Explore | Expand | Enrich

<Codemithra />™

<Codemithra />™

ETHNUS
Explore | Expand | Enrich

SORT WITHOUT EXTRA SPACE

SORT WITHOUT EXTRA SPACE

eTHNUS
Explore | Expand | Enrich

EXPLANATION

- "Sort without extra space" refers to the process of rearranging elements in an array or data structure to achieve a sorted order without using additional memory or storage space to store the sorted values separately. In other words, the sorting algorithm modifies the original array in-place to obtain the sorted result, without allocating memory for a new sorted array.

SORT WITHOUT EXTRA SPACE

EXPLANATION

This approach is particularly useful in scenarios where memory usage is a concern, and you want to minimize the use of extra memory while sorting large datasets. It challenges programmers to design sorting algorithms that can efficiently rearrange elements within the existing data structure, typically an array or a queue, without creating additional copies of the data.

SORT WITHOUT EXTRA SPACE

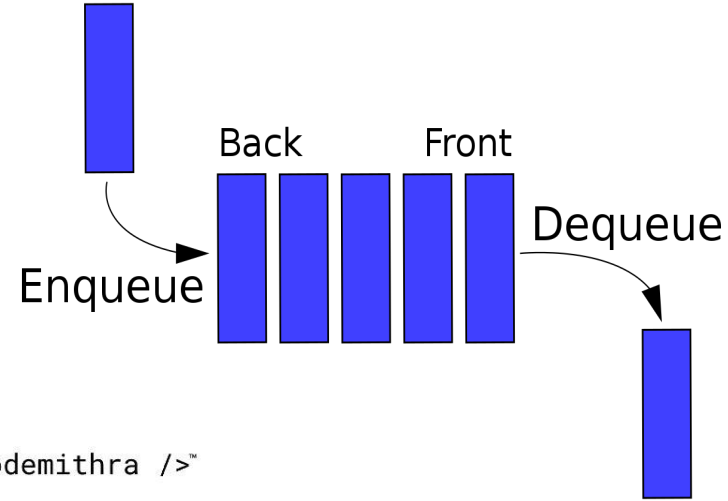
EXPLANATION

where you have a large dataset stored in memory, and you need to sort it without allocating extra memory for a new sorted array. This can save memory and improve efficiency, especially in environments with limited resources.

SORT WITHOUT EXTRA SPACE

EXPLANATION

where you have a large dataset stored in memory, and you need to sort it without allocating extra memory for a new sorted array. This can save memory and improve efficiency, especially in environments with limited resources.



Operation	Description
dequeue	Removes an element from the front of the queue
enqueue	Adds an element to the rear of the queue
first	Examines the element at the front of the queue
isEmpty	Determines whether the queue is empty
size	Determines the number of elements in the queue

EXPLANATION

If we are allowed extra space, then we can simply move all items of queue to an array, then sort the array and finally move array elements back to queue.

Examples

Input

10 -> 7 -> 2 -> 8 -> 6

Output

2 -> 6 -> 7 -> 8 -> 10

SORT WITHOUT EXTRA SPACE

ALGORITHM

Consider the queue made up of two parts, one is sorted and the other is not sorted.

Initially, all the elements are present in not sorted part.

At every step, find the index of the minimum element from the unsorted queue and move it to the end of the queue, that is, to the sorted part.

Repeat this step till all the elements are not present in the sorted queue.

SORT WITHOUT EXTRA SPACE

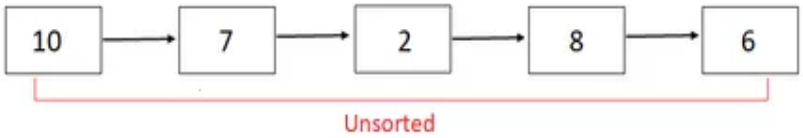
ALGORITHM

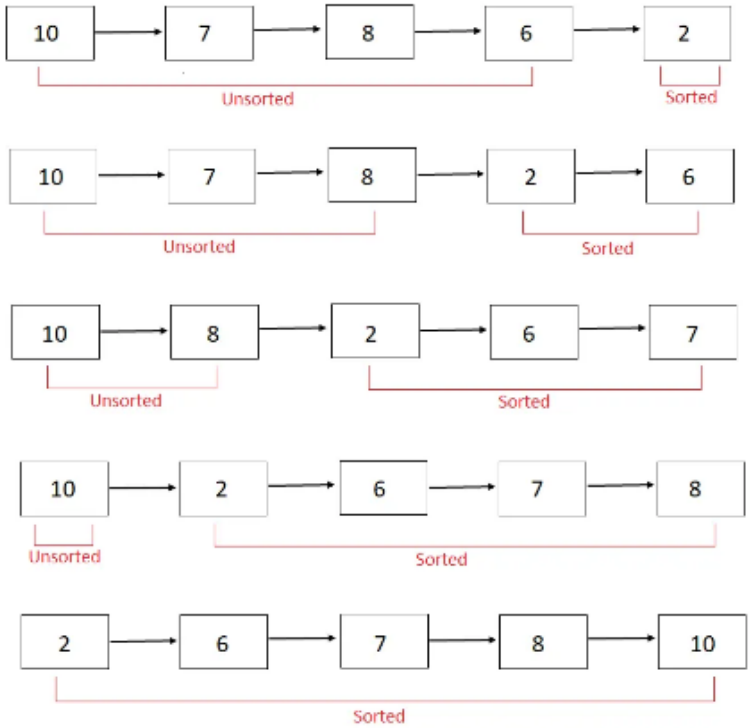
- Run a loop for i = 0 to n(not included), where n is the size of the queue.
- At every iteration initialize minIndex as -1 and minValue as -infinity.
- Run another loop with variable j that finds the minimum element’s index from the unsorted queue.
- The unsorted queue is present from index 0 to index (n – i) for a particular value of i.
- At every iteration, if the current element is less than minValue, update minValue as current element and minIndex as j.
- Traverse in the queue and remove the element at position minIndex and push it at the end of the queue.The queue is sorted, print its elements.

SORT WITHOUT EXTRA SPACE

PSEUDOCODE

At every step, find the index of minimum element in the unsorted part and move it to the end of the queue, that is, sorted part.





SORT WITHOUT EXTRA SPACE

```
import java.util.LinkedList;
import java.util.Queue;
public class SortingAQueueWithoutExtraSpace {
  private static void sortQueue(Queue <Integer>
queue) {
    int n = queue.size();
    for (int i = 0; i < n; i++) {
      int minIndex = -1;
      int minValue = Integer.MAX_VALUE;
      for (int j = 0; j < n; j++) {
        int currValue = queue.poll();
        if (currValue < minValue && j < (n -
i)) {
          minValue = currValue;
          minIndex = j;
        }
        queue.add(currValue);
      }
      // Remove min value from queue
      for (int j = 0; j < n; j++) {
```

```
int currValue = queue.poll();
        if (j != minIndex) {
          queue.add(currValue);
        }
      }
      // Add min value to the end of the queue
      queue.add(minValue);
    }
    // Print the sorted queue
    for (Integer i: queue) {
      System.out.print(i + " ");
    }
    System.out.println();
  }
  public static void main(String[] args) {
    Queue <Integer> q1 = new LinkedList<>();
    q1.add(10);
    q1.add(7);
    q1.add(2);
    q1.add(8);
    q1.add(6);
    sortQueue(q1);
   }}
```

SORT WITHOUT EXTRA SPACE

TIME AND SPACE COMPLEXITY :

Time Complexity: O(n^2) - Quadratic time complexity due to nested loops.

Space Complexity: O(1) - Constant space complexity, as no additional data structures are used proportional to the input size.

1. What is the "Sort Without Extra Space" problem?

Answer: The "Sort Without Extra Space" problem involves sorting an array of integers in ascending order without using additional memory space (constant space complexity).

2. What is the main advantage of sorting without extra space in memory-constrained environments?

Answer: Sorting without extra space is advantageous in memory-constrained environments because it avoids the need for additional memory allocation, making it suitable for systems with limited memory resources.

3. Are there any trade-offs when using in-place sorting algorithms for the "Sort Without Extra Space" problem?

Answer: Yes, there are trade-offs. While in-place sorting algorithms conserve memory, they may have slightly higher time complexity compared to algorithms that use extra space. However, this trade-off is often acceptable when memory is a critical concern.

https://learn.codemithra.com

Explore | Expand | Enrich

THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com